

```

# =====
# BLOB Principle - SAUT 3.6c : SCALING N POUR EXPOSANTS CRITIQUES
# =====
#
# Objectif : mesurer rigoureusement les exposants critiques beta, gamma, nu
# par scaling N. Si les exposants sont independants de N (universalite),
# on a une vraie classe d universalite.
#
# Methode :
#   Pour 4 valeurs de N (100, 200, 400, 800), scan tau autour des transitions
#   identifiees au Saut 3.6b (lecture du rapport prealable).
#   Mesure de PHI (parametre d ordre), chi (susceptibilite), C (chaleur spec).
#   Identification du pic de chi a chaque N -> tau_c(N).
#   Fit des exposants par finite-size scaling (FSS) :
#     - beta : pente de log(PHI) vs log(|tau-tau_c|) en regime ordonne
#     - gamma : pente de log(chi_max) vs log(N) (chi_max ~ N^(gamma/nu))
#     - nu : largeur du pic chi a mi-hauteur ~ N^(-1/nu)
#
# Si les exposants sont stables sur les 4 N, c est une vraie classe.
# Si ils varient, le Blob a un comportement non-FSS (transition non-standard).
# =====

import sys
import os

def auto_check():
    print("=" * 70)
    print("  AUTO-TEST CORRUPTION CARACTERES")
    print("=" * 70)
    try:
        if hasattr(sys, 'argv') and len(sys.argv) > 0 and os.path.exists(sys.argv
            source_file = sys.argv[0]
        else:
            source_file = __file__ if '__file__' in globals() else None
        if source_file and os.path.exists(source_file):
            with open(source_file, 'r', encoding='utf-8') as f:
                content = f.read()
                dangerous = {chr(8216): "U+2018", chr(8217): "U+2019",
                    chr(8220): "U+201C", chr(8221): "U+201D",
                    chr(8211): "U+2013", chr(8212): "U+2014"}
                found = [(name, content.count(c)) for c, name in dangerous.items() if
                    if found:
                        print("ERREUR caracteres corrompus :", found)
                        return False

```

```

        print("OK : pas de corruption")
    else:
        print("Note : auto-test partiel")
except Exception as e:
    print("Note : " + str(e))
print("=" * 70)
return True

if not auto_check():
    sys.exit(1)

import numpy as np
import time
import json
import gc
from collections import Counter

KAPPA_0 = 8.0
BETA_VIAB = 0.30
EPS_DECAL = 0.05

OUTPUT_DIR = '/content/blob_saut36c_results'
os.makedirs(OUTPUT_DIR, exist_ok=True)

class BlobUniverseFixedN:

    def __init__(self, N, tau, seed=0, mode='blob'):
        self.rng = np.random.default_rng(seed)
        self.N = N
        self.tau = tau
        self.mode = mode
        self.seed = seed
        self.positions = [self.rng.random(3) for _ in range(N)]
        self.adj = [set() for _ in range(N)]
        self.deg = np.zeros(N, dtype=np.int32)
        self.edge_list = []
        self.n_tri = 0
        self.n_tetra = 0
        self.S_curr = 0.0
        self._initialize_edges()

    def _initialize_edges(self):
        E_target = int(KAPPA_0 * self.N / 2)
        attempts = 0
        max_attempts = E_target * 100

```

```

while len(self.edge_list) < E_target and attempts < max_attempts:
    u = int(self.rng.integers(self.N))
    v = int(self.rng.integers(self.N))
    attempts += 1
    if u == v or v in self.adj[u]:
        continue
    u, v = min(u, v), max(u, v)
    dt_add = len(self.adj[u] & self.adj[v])
    self.adj[u].add(v)
    self.adj[v].add(u)
    self.deg[u] += 1
    self.deg[v] += 1
    self.edge_list.append((u, v))
    self.n_tri += dt_add
    common = self.adj[u] & self.adj[v]
    if len(common) >= 2:
        cl = list(common)
        for i, w in enumerate(cl):
            for x in cl[i+1:]:
                if x in self.adj[w]:
                    self.n_tetra += 1
self.S_curr = self._compute_S()

def _compute_D(self):
    mu = float(self.deg.mean())
    if mu == 0:
        return 0.0
    return float(self.deg.std()) / mu

def _compute_S(self):
    return np.log(1 + self.n_tri) - BETA_VIAB * self._compute_D()

def _count_tetra_through_edge(self, u, v):
    common = self.adj[u] & self.adj[v]
    if len(common) < 2:
        return 0
    cl = list(common)
    count = 0
    for i, w in enumerate(cl):
        adj_w = self.adj[w]
        for x in cl[i+1:]:
            if x in adj_w:
                count += 1
    return count

def _list_tetrahedra(self):
    result = []

```

```

seen = set()
for u, v in self.edge_list:
    common = self.adj[u] & self.adj[v]
    if len(common) < 2:
        continue
    cl = sorted(common)
    for i, w in enumerate(cl):
        for x in cl[i+1:]:
            if x in self.adj[w]:
                key = tuple(sorted([u, v, w, x]))
                if key not in seen:
                    seen.add(key)
                    result.append(key)
return result

def compute_PHI(self):
    if self.n_tetra == 0:
        return 0.0
    tetras = self._list_tetrahedra()
    if not tetras:
        return 0.0
    edge_to_tetras = {}
    for tet in tetras:
        for i in range(4):
            for j in range(i+1, 4):
                e = (tet[i], tet[j])
                edge_to_tetras.setdefault(e, []).append(tet)
    neighbors_count = {tet: set() for tet in tetras}
    for e, tlist in edge_to_tetras.items():
        if len(tlist) > 1:
            for i, t1 in enumerate(tlist):
                for t2 in tlist[i+1:]:
                    neighbors_count[t1].add(t2)
                    neighbors_count[t2].add(t1)
    total = sum(len(v) for v in neighbors_count.values())
    return float(total) / len(tetras)

def step_swap(self):
    if not self.edge_list:
        return False
    idx = int(self.rng.integers(len(self.edge_list)))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return False
    u_add, v_add = min(u_add, v_add), max(u_add, v_add)

```

```

if (u_add, v_add) == (u_rm, v_rm):
    return False
dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
dtetra_rm = self._count_tetra_through_edge(u_rm, v_rm)
self.adj[u_rm].discard(v_rm)
self.adj[v_rm].discard(u_rm)
self.deg[u_rm] -= 1
self.deg[v_rm] -= 1
dt_add = len(self.adj[u_add] & self.adj[v_add])
self.adj[u_add].add(v_add)
self.adj[v_add].add(u_add)
self.deg[u_add] += 1
self.deg[v_add] += 1
dtetra_add = self._count_tetra_through_edge(u_add, v_add)
new_n_tri = self.n_tri - dt_rm + dt_add
if new_n_tri < 0:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False
new_S = np.log(1 + new_n_tri) - BETA_VIAB * self._compute_D()
delta_S = new_S - self.S_curr

if self.mode == 'null':
    accepted = self.rng.random() < 0.5
else:
    log_alpha = self.N * delta_S / self.tau
    accepted = (log_alpha >= 0) or (self.rng.random() < np.exp(max(log_al

if accepted:
    self.edge_list[idx] = (u_add, v_add)
    self.n_tri = new_n_tri
    self.n_tetra += (dtetra_add - dtetra_rm)
    self.S_curr = new_S
    return True
else:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)

```

```

        self.deg[u_rm] += 1
        self.deg[v_rm] += 1
        return False

def measure_point(N, tau, seed, n_equil, n_measure, mode='blob', n_samples=200):
    sim = BlobUniverseFixedN(N, tau, seed, mode=mode)
    for _ in range(n_equil):
        sim.step_swap()
    sample_interval = max(1, n_measure // n_samples)
    PHI_samples = []
    D_samples = []
    S_samples = []
    for step in range(n_measure):
        sim.step_swap()
        if step % sample_interval == 0:
            PHI_samples.append(sim.compute_PHI())
            D_samples.append(sim._compute_D())
            S_samples.append(sim.S_curr)
    PHI_arr = np.array(PHI_samples)
    D_arr = np.array(D_samples)
    S_arr = np.array(S_samples)
    return {
        'tau': tau, 'seed': seed, 'N': N, 'mode': mode,
        'PHI_mean': float(PHI_arr.mean()),
        'PHI_var': float(PHI_arr.var()),
        'D_mean': float(D_arr.mean()),
        'D_var': float(D_arr.var()),
        'S_mean': float(S_arr.mean()),
        'S_var': float(S_arr.var()),
        'chi': float(N * D_arr.var()),
        'C': float(N * S_arr.var()),
    }

# =====
# Finite-Size Scaling : exposants
# =====
def fit_powerlaw(x, y):
    """Fit log-log  $y = A * x^{\text{slope}}$ . Retourne (slope, R2)."""
    x = np.array(x)
    y = np.array(y)
    valid = (x > 0) & (y > 0)
    if valid.sum() < 3:
        return None, None
    lx = np.log(x[valid])
    ly = np.log(y[valid])

```

```

    slope, _ = np.polyfit(lx, ly, 1)
    pred = np.polyval([slope, _], lx)
    ss_res = np.sum((ly - pred)**2)
    ss_tot = np.sum((ly - ly.mean())**2)
    r2 = 1 - ss_res/ss_tot if ss_tot > 0 else 0
    return float(slope), float(r2)

def find_chi_peak(tau_arr, chi_arr):
    """Trouve le pic de chi par interpolation parabolique sur 3 points autour du
    idx = int(np.argmax(chi_arr))
    if idx == 0 or idx == len(chi_arr) - 1:
        return tau_arr[idx], chi_arr[idx]
    # Interpolation parabolique
    x0, x1, x2 = tau_arr[idx-1], tau_arr[idx], tau_arr[idx+1]
    y0, y1, y2 = chi_arr[idx-1], chi_arr[idx], chi_arr[idx+1]
    denom = (x0-x1)*(x0-x2)*(x1-x2)
    if abs(denom) < 1e-12:
        return tau_arr[idx], chi_arr[idx]
    a = (x2*(y1-y0) + x1*(y0-y2) + x0*(y2-y1)) / denom
    b = (x2*x2*(y0-y1) + x1*x1*(y2-y0) + x0*x0*(y1-y2)) / denom
    if a == 0:
        return tau_arr[idx], chi_arr[idx]
    tau_peak = -b / (2*a)
    chi_peak = y1 + a*(tau_peak - x1)**2 + 0 # approx
    return float(tau_peak), float(chi_peak)

# =====
# CLASSES D UNIVERSALITE
# =====
UNIVERSALITY = {
    'Champ moyen':      {'beta': 0.500, 'gamma': 1.000, 'nu': 0.500, 'gamma_over_n': 1.000},
    'Ising 3D':          {'beta': 0.326, 'gamma': 1.237, 'nu': 0.630, 'gamma_over_n': 1.237},
    'XY 3D':             {'beta': 0.348, 'gamma': 1.316, 'nu': 0.671, 'gamma_over_n': 1.316},
    'Heisenberg 3D':     {'beta': 0.367, 'gamma': 1.388, 'nu': 0.707, 'gamma_over_n': 1.388},
    'Percolation 3D':    {'beta': 0.418, 'gamma': 1.793, 'nu': 0.876, 'gamma_over_n': 1.793},
    'Ising 2D':          {'beta': 0.125, 'gamma': 1.750, 'nu': 1.000, 'gamma_over_n': 1.750}
}

# =====
# MAIN
# =====
print('')
print('=' * 75)
print(' BLOB - SAUT 3.6c - SCALING N POUR EXPOSANTS CRITIQUES')

```

```

print('=' * 75)

# Configuration
# La fenetre tau doit ENCADRER le tau_c approx. detecte au Saut 3.6b.
# Par default : tau_c probable autour de 0.25-0.45, donc fenetre [0.18, 0.50]
TAU_MIN = float(os.environ.get('TAU_MIN', 0.18))
TAU_MAX = float(os.environ.get('TAU_MAX', 0.50))
N_TAU = int(os.environ.get('N_TAU', 12))
N_LIST = [int(x) for x in os.environ.get('N_LIST', '100,200,400,800').split(',')]
N_SEEDS = int(os.environ.get('N_SEEDS', 3))
N_EQUIL = int(os.environ.get('N_EQUIL', 50000))
N_MEASURE = int(os.environ.get('N_MEASURE', 50000))

tau_grid = np.linspace(TAU_MIN, TAU_MAX, N_TAU)

print(f' N a tester : {N_LIST}')
print(f' Grille tau : {TAU_MIN:.3f} a {TAU_MAX:.3f}, {N_TAU} valeurs')
print(f' Seeds par (N, tau) : {N_SEEDS}')
print(f' Equilibration : {N_EQUIL}, mesure : {N_MEASURE}')

# Estimation duree
# Le cout par run scale comme N^2 a cause de PHI cher
estim_min = 0
for N in N_LIST:
    cost_per_step = (N / 200)**1.5 # heuristique
    estim_min += N_TAU * N_SEEDS * (N_EQUIL + N_MEASURE) * 0.000017 * cost_per_st
print(f' Estimation duree : ~{estim_min:.0f} min total')
print('=' * 75)

t_global = time.time()
all_measurements = {}

for N in N_LIST:
    print('')
    print(f' ===== N = {N} =====')
    measurements_N = []
    for tau_idx, tau in enumerate(tau_grid):
        for seed in range(N_SEEDS):
            t0 = time.time()
            m = measure_point(N, tau, seed, N_EQUIL, N_MEASURE, mode='blob')
            measurements_N.append(m)
            elapsed = time.time() - t0
            print(f' N={N} tau={tau:.4f} seed={seed} : PHI={m["PHI_mean"]:.2f}')
            gc.collect()
    all_measurements[N] = measurements_N

# Sauvegarde intermediaire

```

```

interim_path = OUTPUT_DIR + f'/saut36c_interim_N{N}.json'
with open(interim_path, 'w') as f:
    json.dump(measurements_N, f, default=str)
print(f' -> sauvegarde intermediaire {interim_path}')

total_time = time.time() - t_global
print('')
print('=' * 75)
print(f' Toutes mesures terminees en {total_time/60:.1f} min')
print('=' * 75)

# =====
# AGREGATION ET ANALYSE FSS
# =====
profiles_per_N = {}
for N in N_LIST:
    agg = {}
    for m in all_measurements[N]:
        tau = m['tau']
        if tau not in agg:
            agg[tau] = {'PHI': [], 'chi': [], 'C': []}
        agg[tau]['PHI'].append(m['PHI_mean'])
        agg[tau]['chi'].append(m['chi'])
        agg[tau]['C'].append(m['C'])

    tau_sorted = sorted(agg.keys())
    PHI_means = [np.mean(agg[t]['PHI']) for t in tau_sorted]
    chi_means = [np.mean(agg[t]['chi']) for t in tau_sorted]
    C_means = [np.mean(agg[t]['C']) for t in tau_sorted]

    profiles_per_N[N] = {
        'tau': tau_sorted,
        'PHI': PHI_means,
        'chi': chi_means,
        'C': C_means,
    }

# Affichage profils
print('')
print(' --- Profils PHI(tau) par N ---')
header = ' tau'
for N in N_LIST:
    header += f' | PHI(N={N:>4d}) '
print(header)
print(' ' + '-' * (5 + 16 * len(N_LIST)))
for i, t in enumerate(profiles_per_N[N_LIST[0]]['tau']):
    line = f' {t:.3f}'

```

```

    for N in N_LIST:
        if i < len(profiles_per_N[N]['PHI']):
            line += f' | {profiles_per_N[N]["PHI"][i]:>11.3f}'
    print(line)

print('')
print(' --- Profils chi(tau) par N ---')
header = ' tau'
for N in N_LIST:
    header += f' | chi(N={N:>4d})'
print(header)
print(' ' + '-' * (5 + 16 * len(N_LIST)))
for i, t in enumerate(profiles_per_N[N_LIST[0]]['tau']):
    line = f' {t:.3f}'
    for N in N_LIST:
        if i < len(profiles_per_N[N]['chi']):
            line += f' | {profiles_per_N[N]["chi"][i]:>11.5f}'
    print(line)

# =====
# Pic chi -> tau_c(N)
# =====
print('')
print(' --- Pic chi par N ---')
print(f' {"N":>6s} {"tau_c":>8s} {"chi_peak":>10s}')
print(' ' + '-' * 40)
chi_peaks = {}
tau_c_per_N = {}
for N in N_LIST:
    p = profiles_per_N[N]
    tau_c, chi_peak = find_chi_peak(np.array(p['tau']), np.array(p['chi']))
    tau_c_per_N[N] = tau_c
    chi_peaks[N] = chi_peak
    print(f' {N:>6d} {tau_c:>8.4f} {chi_peak:>10.5f}')

# =====
# FSS : exposants
# =====
print('')
print('=' * 75)
print(' FINITE-SIZE SCALING - EXPOSANTS CRITIQUES')
print('=' * 75)

# 1. gamma/nu : chi_max ~ N^(gamma/nu)
N_arr = np.array(list(N_LIST))
chi_peak_arr = np.array([chi_peaks[N] for N in N_LIST])
slope_chi, r2_chi = fit_powerlaw(N_arr, chi_peak_arr)

```

```

print('')
print(f'  chi_peak vs N : chi_max ~ N^(gamma/nu)')
print(f'    Exposant gamma/nu mesure = {slope_chi:.4f}, R2 = {r2_chi:.4f}')

# 2. tau_c(N) : drift -> nu peut etre estime par tau_c(N) - tau_c(infini) ~ N^(-1)
# Si tau_c stable, c est bon signe. Si drift fort, on extrapole.
tau_c_arr = np.array([tau_c_per_N[N] for N in N_LIST])
print('')
print(f'  tau_c par N : {tau_c_arr}')
tau_c_drift = (tau_c_arr.max() - tau_c_arr.min()) / np.mean(tau_c_arr) * 100
print(f'    Drift relatif tau_c : {tau_c_drift:.1f}%')

# 3. beta : a partir de PHI dans le regime ordonne (tau < tau_c)
print('')
print('  Fit beta sur chaque N (PHI ~ |tau - tau_c|^beta) :')
beta_per_N = {}
for N in N_LIST:
    tau_c = tau_c_per_N[N]
    p = profiles_per_N[N]
    # Cote ordonne : tau < tau_c
    mask = np.array(p['tau']) < tau_c
    if mask.sum() < 3:
        continue
    diffs = tau_c - np.array(p['tau'])[mask]
    PHIs = np.array(p['PHI'])[mask]
    slope, r2 = fit_powerlaw(diffs, PHIs)
    if slope is not None:
        beta_per_N[N] = (abs(slope), r2)
        print(f'    N={N} : beta = {abs(slope):.4f}, R2 = {r2:.4f}')

# Stabilite des beta a travers N
if beta_per_N:
    betas = [v[0] for v in beta_per_N.values()]
    beta_mean = np.mean(betas)
    beta_std = np.std(betas)
    beta_cv = beta_std / abs(beta_mean) * 100 if beta_mean != 0 else 0
    print(f'    Stabilite beta : mean={beta_mean:.4f}, std={beta_std:.4f}, CV={be

# =====
# COMPARAISON CLASSES UNIVERSALITE
# =====
print('')
print('=' * 75)
print('  COMPARAISON CLASSES UNIVERSALITE')
print('=' * 75)
print('')
print(f'  Blob mesure : gamma/nu = {slope_chi:.3f}')

```

```

if beta_per_N:
    print(f'                                beta = {beta_mean:.3f} (CV {beta_cv:.1f}%)')

print('')
print(f' {"Classe":18s} {"beta":>7s} {"gamma/nu":>9s} {"ecart_b":>8s} {"ecar
print(' ' + '-' * 75)
best_match = None
best_score = float('inf')
for class_name, exps in UNIVERSALITY.items():
    err_beta = abs(beta_mean - exps['beta']) if beta_per_N else float('inf')
    err_gn = abs(slope_chi - exps['gamma_over_nu']) if slope_chi else float('inf')
    score = err_beta**2 + err_gn**2
    flag = ''
    if score < 0.01:
        flag = ' *** MATCH ***'
    elif score < 0.05:
        flag = ' * proche *'
    print(f' {class_name:18s} {exps["beta"]:>7.3f} {exps["gamma_over_nu"]:>9.3
    if score < best_score:
        best_score = score
        best_match = class_name

print('')
print('=' * 75)
print(f' VERDICT : meilleur match = {best_match} (score {best_score:.4f})')
if best_score < 0.01:
    print(' >> CLASSE D UNIVERSALITE IDENTIFIEE')
elif best_score < 0.05:
    print(' >> Match approche avec ' + best_match)
else:
    print(' >> Le Blob a probablement une classe d universalite propre')

# =====
# SAUVEGARDE
# =====
report = {
    'config': {
        'N_list': N_LIST,
        'tau_grid': list(tau_grid),
        'n_seeds': N_SEEDS,
        'n_equil': N_EQUIL,
        'n_measure': N_MEASURE,
        'total_time_s': total_time,
    },
    'profiles_per_N': profiles_per_N,
    'tau_c_per_N': tau_c_per_N,
    'chi_peaks': chi_peaks,

```

```
    'gamma_over_nu': slope_chi,  
    'gamma_over_nu_r2': r2_chi,  
    'beta_per_N': {str(N): list(v) for N, v in beta_per_N.items()},  
    'best_match': best_match,  
    'best_score': best_score,  
}
```

```
report_path = OUTPUT_DIR + '/saut36c_RAPPORT.json'  
with open(report_path, 'w') as f:  
    json.dump(report, f, indent=2, default=str)  
print('')  
print(f' Rapport : {report_path}')print(f' Taille : {os.path.getsize(report_path)} octets')
```